

Tool Fundamentals & Design Principles

1. Why Great Tools = Foundation of Agents

Tools are the **interface** between LLMs and the real world. Even the most intelligent model fails without well-designed tools.

Key Insight:

- 70-80% of agent reliability comes from tool quality.
- Bad tools → hallucinations, crashes, security issues.
- Great tools → predictable, maintainable, production-ready agents.

Real-World Impact Examples:

- Search tool for research agents
 - Database query tool for data analysis
 - Queue explorer for automation testing (your use case)
-

2. Function Calling / Tool Use Basics

What is Function Calling?

Modern LLMs (OpenAI, Anthropic, Groq, etc.) can call external functions/tools instead of just generating text.

Basic Flow:

1. User query → LLM
2. LLM decides to call tool(s)
3. System executes tool
4. Tool result → back to LLM
5. LLM generates final response

Core Components:

- Tool definition (schema + description)
- Tool execution
- Response parsing

Simple Example (Python + LangChain):

```
from langchain_core.tools import tool
from pydantic import BaseModel, Field

@tool
def add_numbers(a: int, b: int) -> int:
    """Add two numbers and return the result."""
    return a + b
```

3. Tool Schemas: JSON Schema vs Pydantic

LLMs receive tools as **JSON Schema**. We write them using **Pydantic** for developer convenience.

JSON Schema (what LLM sees):

```
{
  "name": "add_numbers",
  "description": "Add two numbers...",
  "parameters": {
    "type": "object",
    "properties": {
      "a": {"type": "integer"},
      "b": {"type": "integer"}
    },
    "required": ["a", "b"]
  }
}
```

Pydantic (what we write):

```
class AddNumbersInput(BaseModel):
    a: int = Field(..., description="First number")
    b: int = Field(..., description="Second number")

@tool(args_schema=AddNumbersInput)
def add_numbers(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b
```

Advantages of Pydantic:

- Automatic schema generation
 - Runtime validation
 - Type hints
 - Excellent LLM-readable descriptions via Field()
-

4. How LLMs Decide Which Tool to Call

The LLM treats tool descriptions like **highly important prompt text**.

Decision Process:

1. Understand user intent
2. Read all tool descriptions carefully
3. Calculate relevance (probabilistic)
4. Choose best tool(s) – can call multiple in parallel

Pro Tip: Write docstrings as if explaining to a very smart but literal colleague.

Good Description Example:

```
@tool
def search_web(query: str, num_results: int = 5) -> list[dict]:
    """Search the web for up-to-date information.

    Use this when user needs current events, facts, or research.
    Returns list of results with title, url, and snippet.
    """
```

5. Parsing Tool Calls & Graceful Error Handling

```
# Example response handling
def handle_tool_call(response):
    if response.tool_calls:
        for tool_call in response.tool_calls:
            try:
                result = execute_tool(tool_call)
                # Feed result back to LLM
            except Exception as e:
```

```
# Graceful fallback
result = f"Tool failed: {str(e)}. Trying alternative appr
return result
```

Best Practices:

- Always wrap tool execution in try/except
 - Return structured error information
 - Implement retries with exponential backoff
 - Never expose raw exceptions to the LLM unless sanitized
-

6. Tool Design Principles

Principle 1: One Tool = One Job (Single Responsibility)

- Keep tools focused and small
- Easier for LLM to understand and use correctly

Bad: `process_data_and_send_email()`

Good: `process_data()` + `send_email()`

Principle 2: Clear, LLM-Readable Docstrings

- Be specific and example-rich
- Include when to use and when not to use
- Mention return format

Principle 3: Always Return Structured Data

```
class ToolResult(BaseModel):
    success: bool
    data: dict | list | None
    error: str | None = None

# Always return ToolResult model
```

Never return plain strings — it breaks structured reasoning.

Principle 4: Put Resilience Inside Tools

- Input validation
- Fallback logic
- Timeouts
- Rate limiting
- Sanitization

Keep agent logic clean.

7. Live Coding Examples

1. Basic Tool with @tool

```
from langchain_core.tools import tool

@tool(parse_docstring=True)
def multiply(a: float, b: float) -> float:
    """Multiply two numbers.

    Useful for any mathematical calculation requiring multiplication.
    """
    return a * b
```

2. Tool with Pydantic Models

```
from pydantic import BaseModel, Field
from langchain_core.tools import StructuredTool

class SearchInput(BaseModel):
    query: str = Field(..., description="Search query")
    max_results: int = Field(5, description="Maximum results to return")

def search_implementation(query: str, max_results: int):
    # implementation
    return {"results": [...]}

search_tool = StructuredTool.from_function(
    func=search_implementation,
    name="search_web",
    description="...",
    args_schema=SearchInput
)
```

3. Error Handling Wrapper

```
def robust_tool(func):
    def wrapper(*args, **kwargs):
        try:
            return func(*args, **kwargs)
        except Exception as e:
            return {"success": False, "error": str(e)}
    return wrapper
```

8. Common Anti-Patterns & Pitfalls

1. Vague docstrings → LLM picks wrong tool
 2. Returning free text → breaks agent reasoning
 3. No error handling → agent crashes
 4. Tools doing multiple jobs
 5. Missing type hints / schemas
 6. Hard-coded values instead of parameters
-

9. Real-World Use Cases

- **Search Tool:** Web + vector search hybrid
 - **Database Tool:** Read-only queries with sanitization
 - **File Processor:** Safe file operations
 - **Queue Explorer:** For automation testing (dead letter analysis)
-

Homework

Build **3 production-ready tools**:

1. A calculator/math tool suite (add, multiply, etc.)
2. A web search or API calling tool
3. One tool relevant to your project (e.g., queue inspection)

NOTE: Apply all 4 design principles.

References & Further Reading

- [LangChain Tool Documentation](#)
- [Pydantic Documentation](#)
- [OpenAI Function Calling Guide](#)